

P1249R0

std::forward from std::initializer_list

Published Proposal, 2018-10-03

This version:

<http://wg21.link/P1249r0>

Author:

[Alex Christensen](#) (Apple)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

github.com/achristensen07/papers/blob/master/source/P1249r0.bs

Abstract

Loosen const requirements of std::initializer_list to make it usable by non-copyable types.

Table of Contents

- 1 Introduction and motivation:**
- 2 Proposed changes:**
- 3 Compatibility considerations:**
- 4 Clang implementation notes:**

§ 1. Introduction and motivation:

I often find myself writing templates that use `std::initializer_list` in the constructor, like this:

```
#include <memory>
#include <initializer_list>

template<typename T>
class Vector {
public:
    Vector(std::initializer_list<T>&& list)
        : buffer(std::make_unique<T[]>(list.size()))
    {
        for (auto&& element : list)
            buffer[size++] = element;
    }
private:
    size_t size { 0 };
    std::unique_ptr<T[]> buffer;
};

void currentlyCompiles() {
    Vector<int> v1({ 1, 2 });
}
```

However, when I want to use non-copyable types, I can't:

```
void sadness() {
    // error: object of type 'std::__1::unique_ptr<int, std::__1::default_delete<int>>'
    // cannot be assigned because its copy assignment operator is implicitly deleted
    //         buffer[size++] = element;
    Vector<std::unique_ptr<int>> v2({ std::make_unique<int>(3), std::make_unique<int>(4) });
}
```

This seems like something `std::forward` ought to solve, but if we add a `std::forward<T>` to the assignment in Vector's constructor, we get different errors:

```

error: no matching function for call to 'forward'
      buffer[size++] = std::forward<T>(element);
                        ^~~~~~
note: in instantiation of member function 'Vector<std::__1::unique_ptr<int,
      Vector<std::unique_ptr<int>> v2({ std::make_unique<int>(3), std::make_ur
      ^
note: candidate function not viable:
      1st argument ('const std::__1::unique_ptr<int, std::__1::default_dele
forward(typename remove_reference<_Tp>::type& __t) _NOEXCEPT
      ^
note: candidate function not viable:
      1st argument ('const std::__1::unique_ptr<int, std::__1::default_dele
forward(typename remove_reference<_Tp>::type&& __t) _NOEXCEPT

```

These errors are caused by the current definition of `std::initializer_list`, which defines its non-const iterators to be const. There is no direct workaround. In order to get `initializer_list`-like behavior, we currently have to use variadic templates:

```

#include <memory>
#include <initializer_list>

template<typename T>
class Vector {
public:
    Vector(std::initializer_list<T>&& list)
        : buffer(std::make_unique<T[]>(list.size()))
    {
        for (auto&& element : list)
            buffer[size++] = element;
    }
    template<typename... Elements>
    static Vector<T> createFrom(Elements&&... elements)
    {
        Vector<T> vector;
        vector.size = sizeof...(elements);
        vector.buffer = std::make_unique<T[]>(vector.size);
        vector.initialize<0>(std::forward<Elements>(elements)...);
        return vector;
    }
private:
    Vector() = default;
    template<size_t index, typename Element, typename... RemainingElements>
    void initialize(Element&& item, RemainingElements&&... remainingElement
    {
        initialize<index>(std::forward<Element>(item));
        initialize<index + 1>(std::forward<RemainingElements>(remainingElen
    }
    template<size_t index, typename Element>
    void initialize(Element&& value)
    {
        buffer[index] = std::forward<Element>(value);
    }
    size_t size { 0 };
    std::unique_ptr<T[]> buffer;
};

void currentlyCompilesButUgly() {
    Vector<int> v1({ 1, 2 });
    auto v2 = Vector<std::unique_ptr<int>>::createFrom(
        std::make_unique<int>(3),
        std::make_unique<int>(4)

```

```
);  
}
```

Not only does this seem excessive in the definition of my Vector, but it also requires strange syntax when using my Vector with non-copyable types. But I would like to write code that uses `std::initializer_list` for all constructors, like this:

```
#include <memory>  
#include <initializer_list>  
  
template<typename T>  
class Vector {  
public:  
    Vector(std::initializer_list<T>&& list)  
        : buffer(std::make_unique<T[]>(list.size()))  
    {  
        for (auto&& element : list)  
            buffer[size++] = std::forward<T>(element);  
    }  
private:  
    size_t size { 0 };  
    std::unique_ptr<T[]> buffer;  
};  
  
void elegant() {  
    Vector<int> v1({ 1, 2 });  
    Vector<std::unique_ptr<int>> v2({ std::make_unique<int>(3), std::make_ur  
}
```

This paper proposes changes to make this possible.

§ 2. Proposed changes:

Section 16.10.1 should remove `const` as follows:

```

namespace std {
template<class E> class initializer_list {
public:
using value_type = E;
using reference = const E&;
using const_reference = const E&;
using size_type = size_t;
using iterator = const E*;
using const_iterator = const E*;
constexpr initializer_list() noexcept;
constexpr size_t size() const noexcept;
constexpr const E* begin() const noexcept; // first element
constexpr const E* end() const noexcept; // one past the last element
};
// 16.10.4, initializer list range access
template<class E> constexpr const E* begin(initializer_list<E> il) noexcept;
template<class E> constexpr const E* end(initializer_list<E> il) noexcept;
}

```

Section 16.10.3 should remove const as follows:

```
constexpr const E* begin() const noexcept;
```

...

```
constexpr const E* end() const noexcept;
```

Section 16.10.4 should remove const as follows:

```
template<class E> constexpr const E* begin(initializer_list<E> il) noexcept;
```

...

```
template<class E> constexpr const E* end(initializer_list<E> il) noexcept;
```

§ 3. Compatibility considerations:

This change does not break binary compatibility, but it could possibly break source compatibility with code that does tricky things with types. Consider the following example:

```
#include <memory>
#include <iostream>
#include <initializer_list>

template<typename T>
class Vector {
public:
    Vector(std::initializer_list<T>&& list)
        : buffer(std::make_unique<T[]>(list.size()))
    {
        for (auto&& element : list) {
            checkConst(element); // Calls a different function
            buffer[size++] = element;
        }
    }
private:
    void checkConst(T&) { std::cout << "non-const" << std::endl; }
    void checkConst(const T&) { std::cout << "const" << std::endl; }

    size_t size { 0 };
    std::unique_ptr<T[]> buffer;
};

int main() {
    static_assert(std::is_const<typename std::remove_reference<typename std::
    Vector<int> v1({ 1, 2 }));
}
```

In practice, most uses of `std::initializer_list` do not do these things, and most const overloads used with `std::initializer_list` do the same thing as the non-const version. I believe compatibility issues with this change will be minimal and worth the benefits of future `std::initializer_list` use. In order to mitigate these compatibility concerns, we could add a feature test macro.

§ 4. Clang implementation notes:

In addition to changing to the `initializer_list` header, clang currently requires loosening of a check in `AggExprEmitter::VisitCXXStdInitializerListExpr` in `CGExprAgg.cpp` or it will fail with this error:

```
error: cannot compile this weird std::initializer_list yet
```